ML ENGINEERING PROJECT BLUEPRINT

Multimodal Image & Text Semantic Search Engine

A production-oriented retrieval system built on CLIP/SigLIP embeddings, PostgreSQL + pgvector, and FastAPI — from a Flickr30k prototype to an e-commerce-ready architecture.

PREPARED FOR	Soufian — Data Science & AI Engineering
DOCUMENT TYPE	End-to-end project plan & portfolio blueprint
PRIMARY DATASET	Flickr30k (prototype) → ABO (production path)
CORE STACK	PyTorch · Hugging Face · PostgreSQL/pgvector · FastAPI · Docker
SCOPE	Data engineering → Modeling → API → MLOps → Deployment

MVP-first design

Evaluation-driven

Student-budget deployment

Table of Contents

01 Executive Summary

02 Business & Real-World Use Case

03 Functional Requirements

04 Non-Functional Requirements

05 Dataset Analysis — Flickr30k

06 Data Engineering Pipeline

07 Exploratory Data Analysis

08 Preprocessing Strategy

09 Model Strategy

10 Embedding Generation Pipeline

11 Database Design (PostgreSQL + pgvector)

12 Retrieval System

13 Multimodal Search Strategy

14 Evaluation Dataset Construction

15 Evaluation Metrics

16 Baseline Experiments

17 Model Improvement Strategy

18 Two-Stage Re-Ranking

19 API Design

20 Frontend

21 Repository Structure

22 Testing Strategy

23 MLOps (MLflow, Versioning, Reproducibility)

24 CI/CD

25 Monitoring & Observability

26 Security

27 Deployment Architecture

28 Performance Optimization

29 Failure Cases & Diagnostics

30 Advanced Version Roadmap

31 Transition: Flickr30k → E-Commerce (ABO)

32 Project Milestones

33 Week-by-Week Implementation Plan

34 Learning Requirements

35 Final Portfolio Presentation

36 Interview Preparation

37 Final A-to-Z Checklist

01 Executive Summary

What the project does

This project is a **multimodal semantic search engine**: a system that lets a user search a collection of images using a text query, a reference image, or a combination of both, and receive a ranked list of the most semantically relevant images with similarity scores. Internally, images and text are projected into a single shared vector space using a pretrained vision-language model (CLIP or SigLIP), stored in a relational database extended with vector search (PostgreSQL + pgvector), and retrieved through nearest-neighbor search that is wrapped in a versioned, tested, monitored API and deployed as a real running service.

Why this is an ML Engineering project, not a notebook demo

> **THE CORE DISTINCTION**
>
> Calling `CLIP.encode()` on a folder of images and computing cosine similarity in a notebook is a valid *proof of concept*, but it is not an engineered system. This project treats the model as one component inside a larger pipeline that must be reproducible, testable, observable, and deployable. The engineering work — not the model call — is what differentiates a junior ML engineer's portfolio from a tutorial follow-along.

Notebook demo	This project (engineered system)
Loads images from a local folder, no validation	Ingestion pipeline with integrity checks, corruption detection, deduplication, schema validation
Embeddings recomputed every run, kept in memory	Embeddings persisted in PostgreSQL/pgvector, versioned by model name and hash, never recomputed unnecessarily
Similarity computed with a brute-force loop over a NumPy array	Indexed approximate nearest-neighbor search (HNSW) exposed through a query layer with pagination and score fusion
No evaluation beyond "it looks right"	Formal evaluation set with Recall@K, MRR, NDCG, and tracked experiments in MLflow
No API, no tests, no deployment	FastAPI service, pytest suite (unit/integration/API), Dockerized, CI/CD, deployed and monitored

What ML problems it solves

- **Cross-modal representation learning** — aligning images and text in one embedding space.
- **Information retrieval** — approximate nearest-neighbor search at scale over dense vectors.
- **Ranking** — ordering candidates by relevance, optionally through a second, more expensive re-ranking stage.
- **Multimodal fusion** — combining signals from two modalities (image + text) into one relevance score.
- **Evaluation of unsupervised/pretrained retrieval** — measuring the quality of a system that was never explicitly trained on your dataset.

Why pretrained models are the right starting point

CLIP and SigLIP were trained on hundreds of millions to billions of image-text pairs — orders of magnitude more data and compute than a student can access. Training a comparable vision-text encoder from scratch is neither feasible nor the point of this project. The engineering value is in **correctly integrating, evaluating, and**

productionizing a strong pretrained encoder, then improving it surgically (fine-tuning, re-ranking) only where evaluation shows it is worth the cost. This mirrors how real ML teams work: pretrained foundation models plus domain-specific engineering on top.

What the final application looks like

A deployed web application with a search bar, an image upload button, and a results grid. A user can type "two dogs playing outside," upload a photo of a similar scene, or do both at once, and receive a ranked grid of images with captions and similarity scores in well under a second. Behind that simple interface sits a documented API, a versioned model, a queryable vector database, automated tests, CI/CD, and basic production monitoring.

Skills demonstrated

Category	Concrete evidence in this project
Data engineering	Ingestion, validation, dedup, corruption handling, reproducible splits
Computer vision & NLP	Image preprocessing, tokenization, encoder usage, embedding normalization
Multimodal ML	Shared embedding space, fusion strategies for image+text queries
Information retrieval	ANN search, HNSW/IVFFlat indexing, Recall@K/MRR/NDCG evaluation
MLOps	MLflow experiment tracking, model/data versioning, drift monitoring
Backend engineering	FastAPI service design, request/response contracts, error handling
DevOps	Docker, Docker Compose, GitHub Actions CI/CD, deployment
Software quality	Unit/integration/API test suites, structured logging
System design	Two-stage retrieval, caching, scalability tradeoffs, cost-aware architecture

What should appear on a GitHub portfolio

- A README that opens with the live demo link and a 10-second GIF of the search UI in action.
- An architecture diagram (the one built in Section 27) rendered directly in the README.
- A results table: Recall@1/5/10, MRR, and P50/P95 latency, compared across at least two baselines.
- Clear "how to run locally" instructions using Docker Compose, and a link to the API docs.
- An explicit "Design decisions and tradeoffs" section — this is what signals engineering maturity to reviewers, more than the model choice itself.

HOW TO READ THIS DOCUMENT

Every component below is tagged `MVP` , `RECOMMENDED` , or `ADVANCED` . Build MVP first, get it evaluated and deployed, then layer in RECOMMENDED items as time allows. ADVANCED items are explicitly out of scope for the first deployed version — they exist so you know where the project can grow, for later iterations or interview conversations.

02 Business & Real-World Use Case

Semantic image-text retrieval is not an academic toy problem — it is the core technology behind several product categories. Understanding these use cases is what lets you talk about the project convincingly in an interview, beyond "I called CLIP."

Use case	What it needs from this architecture	Real examples
Visual product search	Image → image retrieval, fast ANN search, product metadata fusion	Pinterest Lens, Google Lens shopping, ASOS visual search
Text-based product discovery	Text → image retrieval with attribute filtering	"red midi dress with long sleeves" on an e-commerce site
Content moderation / discovery	Fast similarity search for near-duplicate or related content	Stock photo platforms (Shutterstock, Unsplash search)
Digital asset management (DAM)	Natural-language search over unlabeled internal image libraries	Media companies, marketing teams searching brand asset libraries
Recommendation ("more like this")	Nearest-neighbor retrieval used as a candidate generator upstream of a ranker	"Similar items" rails on e-commerce and media platforms

From Flickr30k prototype to e-commerce product search

Flickr30k is used here purely to **de-risk the architecture**: it is small enough to iterate on quickly, has clean natural-language captions that make embedding quality easy to reason about, and has no licensing complexity for a portfolio project. Every component built against Flickr30k — ingestion, database schema, embedding pipeline, retrieval API, evaluation harness — is dataset-agnostic by design. Section 31 details exactly what changes (and what stays identical) when swapping in a product catalog such as Amazon Berkeley Objects (ABO): richer metadata (titles, bullet points, structured attributes, price, category) replaces free-form captions, and the same embedding + pgvector + FastAPI stack is reused with almost no architectural change. This transition story — "I validated the architecture on a research dataset before pointing it at a production-shaped one" — is itself a strong engineering narrative for interviews.

03 Functional Requirements

A. Text → Image Search MVP

User submits a free-text query (e.g. "two dogs playing outside"). The system encodes the query with the text encoder, searches the image-embedding index for the closest vectors, and returns the Top-K images ranked by cosine similarity.

B. Image → Image Search MVP

User uploads an image. The system encodes it with the image encoder and searches the same (or a separate) image-embedding index for the closest vectors, returning visually/semantically similar images — excluding the query image itself if it exists in the index.

C. Image + Text → Image Search RECOMMENDED

User uploads an image and supplies a text modifier (e.g. image of a dog + text "in a park"). The system fuses both embeddings (see Section 13) into a single query representation and retrieves images matching both constraints. This is the hardest functional requirement and is explicitly recommended rather than MVP — ship text-only and image-only search first, evaluate them, then add fusion.

D. Result Ranking MVP

Every result returned to the client includes: the image (URL or reference), its internal image ID, its associated caption(s), a similarity score in [0, 1] or [-1, 1] depending on the metric, and its rank position (1-indexed) in the current result set.

E. API Endpoints MVP

Endpoint	Method	Purpose	Priority
/search/text	POST	Text query → ranked images	MVP
/search/image	POST	Image upload → ranked images	MVP
/search/multimodal	POST	Image + text → ranked images	RECOMMENDED
/health	GET	Liveness/readiness check (DB connectivity, model loaded)	MVP
/model/info	GET	Currently served model name, version, embedding dimension	MVP

Request/response contract — /search/text

Field	Location	Type	Notes
query	request body	string	1–500 characters, required
top_k	request body	integer	default 10, max 100
results[].image_id	response body	string/UUID	internal identifier

results[].image_url	response body	string	path or public URL
results[].caption	response body	string	best/first associated caption
results[].score	response body	float	similarity score
results[].rank	response body	integer	1-indexed position
model_version	response body	string	which encoder produced these embeddings
latency_ms	response body	float	server-side processing time, for transparency and debugging

/search/image follows the same response shape, with the request being a multipart file upload instead of a JSON query string. */search/multimodal* accepts both an optional file and an optional query string, plus a fusion weight parameter (alpha), and requires at least one of the two modalities to be present.

04 Non-Functional Requirements

Dimension	Realistic student-portfolio target	Why this target
Latency (text search)	P50 < 300ms, P95 < 800ms on CPU, single request	Achievable on CPU inference for a ~150M parameter encoder with a pre-warmed index
Latency (image search)	P50 < 500ms, P95 < 1200ms	Includes upload decode + preprocessing + encode + search
Reproducibility	Any embedding regenerated from the same model+preprocessing config must match to floating-point tolerance	Required for trustworthy evaluation and for auditability
Scalability	Index must remain sub-second up to ~50k–100k vectors on a single small VM	Covers Flickr30k (31k images) with headroom; beyond this, sharding/ANN tuning matters more
Reliability	/health responds correctly even under DB restart; API returns typed errors instead of crashing	Baseline production hygiene
Testability	>70% coverage on core retrieval/embedding logic	Enough to catch regressions without chasing 100% vanity coverage
Security	Upload size/type validation, no secrets in source control, parameterized SQL	Minimum bar for any public-facing demo
Logging & observability	Structured logs for every request (latency, endpoint, status, model version)	Enables debugging without reproducing issues manually
Maintainability	Config-driven (model name, DB URL, thresholds) — no hardcoded values in logic code	Lets you swap CLIP↔SigLIP or Flickr30k↔ABO without rewriting code

05 Dataset Analysis — Flickr30k

Property	Value
Total images	≈ 31,783 images
Captions per image	5 independent human-written captions per image
Total captions	≈ 158,900
Source	Flickr photo-sharing site, curated for research use
Typical distribution size (images)	≈ 4–5 GB uncompressed
License	Custom research/academic license (non-commercial) — see Kaggle/original UIUC page; state this explicitly in the README and do not claim commercial rights
Standard splits	No official train/val/test split ships with the raw data — commonly people use the "Karpathy split" (≈29,000 train / 1,000 val / 1,000 test) which you should replicate for comparability with published numbers

Metadata to store per image

- Original filename and stable internal `image_id`
- File size (bytes), width, height, aspect ratio, color mode (RGB/L/CMYK)
- SHA-256 hash of file content (for dedup and integrity verification)
- Associated caption IDs and raw caption text
- Split assignment (train/val/test)
- Ingestion timestamp and dataset version tag

Dataset strengths

- Small enough to fully process on a laptop CPU in minutes, ideal for fast iteration.
- Multiple independent captions per image give a natural way to build an evaluation set and to study caption variability.
- Well-studied in the retrieval literature, so published Recall@K numbers exist as a rough sanity check for your own baseline.

Dataset limitations — be explicit about these in the README

KNOWN LIMITATIONS

- Everyday/generic scenes (people, animals, outdoor activities) — not representative of product images, documents, or specialized domains.
- Captions describe general scene content, not fine-grained attributes (color, material, brand) that e-commerce search needs.
- No structured metadata (category, price, attributes) — this is exactly the gap Section 31 addresses when moving to ABO.
- Some demographic and geographic bias inherited from Flickr's user base circa the dataset's collection period.

> • Non-commercial license — fine for a portfolio/demo, not for a real product without re-licensing the data.

Directory layout

data/

- **raw/ — untouched downloaded archive + original captions file**
- **images/ — validated, deduplicated images, organized by split**
- **processed/ — cleaned captions table, computed embeddings cache, train/val/test manifests**
- **metadata/ — data validation reports, dataset statistics, dataset version manifest (hash + row counts)**

Nothing in `raw/` is ever modified in place — this is what makes the pipeline reproducible and re-runnable from scratch.

06 Data Engineering Pipeline

Download raw archive (images.zip + captions.token file)

↓

Integrity check — file counts, checksums against a manifest

↓

Corrupted image detection — attempt decode of every file

↓

Duplicate detection — hash-based (exact) and perceptual-hash-based (near-duplicate)

↓

Missing-image detection — captions with no matching image file, and vice versa

↓

Caption cleaning — encoding fixes, whitespace normalization, empty-caption removal

↓

Metadata validation — schema check on every record (types, ranges, required fields)

↓

Deterministic train/val/test split (fixed seed, reproducible on re-run)

↓

Validated dataset + data-quality report written to metadata/

Data quality tests — examples of rules to enforce

Check	Rule	Action on failure
Image decodability	Every image file must open and decode without exception	Move to `quarantine/`, log, exclude from dataset
Minimum resolution	Width and height both ≥ 32px	Exclude, log reason
Exact duplicates	SHA-256 hash collisions	Keep one copy, record the mapping
Near-duplicates	Perceptual hash Hamming distance below a threshold	Flag for manual review, do not auto-delete
Caption completeness	Every image has 1–5 non-empty captions	Flag images with 0 captions as unusable for supervised evaluation
Caption length	Between 3 and 60 tokens	Flag outliers for manual inspection, do not silently drop

Encoding	Valid UTF-8, no control characters	Auto-repair where possible, else quarantine
Split leakage	No image_id appears in more than one split	Hard failure — halts the pipeline

Reproducibility requirements

- All randomness (splitting, sampling) uses a fixed, logged seed.
- The pipeline writes a dataset version manifest: row counts, file hashes, seed used, and pipeline code version (git commit hash).
- Re-running the pipeline on the same raw input must produce byte-identical processed outputs.

07 Exploratory Data Analysis

Investigation	Recommended plot	What it should reveal
Image dimensions	Scatter of width vs. height, histogram of both	Whether resizing to a fixed square will distort many images (aspect-ratio skew)
Aspect ratios	Histogram of width/height ratio	How much letterboxing/cropping the preprocessing step will need to handle
Caption length	Histogram of token count per caption	Whether a max-length truncation in the tokenizer will cut off meaningful content
Vocabulary	Word-frequency bar chart (top 30), vocabulary size vs. corpus size curve	Domain vocabulary breadth; rare-word long tail that pretrained tokenizers may split into many subword pieces
Missing data	Bar chart of missing-field counts per data source	Confirms ingestion validity before embeddings are generated on top of it
Duplicate images	Table/count of exact and near-duplicates found	Risk of retrieval results being dominated by near-identical images
Duplicate captions	Count of captions repeated verbatim across different images	Generic captions ("a dog running") reduce the discriminative power of text queries
Image–caption relationship	Manual spot-check grid: N random images with their 5 captions overlaid	Qualitative sanity check that captions genuinely describe image content
File size distribution	Histogram of file sizes in KB	Storage planning and outlier detection (unusually large/small files often indicate corruption)

DELIVERABLE

A single EDA notebook that outputs a static HTML/PDF report with all plots above plus a one-paragraph written interpretation per plot. This artifact belongs in `notebooks/`, separate from production code, and is linked from the README as evidence of data understanding.

08 Preprocessing Strategy

Image preprocessing

Step	Detail
RGB conversion	Force all images (including grayscale/CMYK/RGBA) to 3-channel RGB before encoding
Resizing	Resize to the encoder's expected input size (e.g. 224×224 for many CLIP variants), following the encoder's own documented resize+crop recipe rather than inventing a custom one
Normalization	Use the exact mean/std the pretrained encoder was trained with — mismatched normalization silently degrades embedding quality without throwing an error
Corrupted image handling	Caught upstream in the data engineering pipeline (Section 6); at inference time, corrupted uploads are rejected with a clear 4xx error, never silently skipped

Text preprocessing

For captions and search queries, preprocessing is intentionally minimal: strip leading/trailing whitespace, normalize Unicode, and enforce a maximum character/token length matching the encoder's tokenizer limit. That is essentially the entire pipeline.

> **WHY AGGRESSIVE NLP PREPROCESSING HURTS CLIP/SIGLIP-STYLE MODELS**
>
> Classical NLP pipelines (lowercasing, stopword removal, stemming/lemmatization, punctuation stripping) were designed for bag-of-words and TF-IDF style models that have no notion of word order or context. CLIP/SigLIP text encoders are transformer language models pretrained on natural, un-preprocessed captions — they rely on function words, capitalization, and word order for context. Removing stopwords turns "a dog playing in the park" into "dog playing park," which shifts the sentence embedding away from the distribution the encoder was actually trained on, typically *reducing* retrieval quality. The correct preprocessing philosophy here is: preprocess for validity (encoding, length), not for "cleanliness" in the classical NLP sense.

09 Model Strategy

Baseline model MVP

Use a pretrained CLIP checkpoint (e.g. OpenAI ViT-B/32 or a comparable Hugging Face checkpoint) as the baseline. It has a well-understood image encoder (Vision Transformer) and text encoder (Transformer), both projecting into a shared embedding space of fixed dimensionality (commonly 512 or 768), with embeddings typically L2-normalized before similarity comparison so that cosine similarity and dot product become equivalent.

Concept	Explanation
Image encoder	Vision Transformer (or ResNet in older CLIP variants) that maps an image to a fixed-length vector
Text encoder	Transformer that maps a tokenized caption/query to a fixed-length vector
Shared embedding space	Both encoders are trained (contrastively) so that matching image-text pairs land close together and non-matching pairs land far apart
Embedding dimension	Fixed per checkpoint (e.g. 512 for ViT-B/32); must be stored consistently in the database schema
Normalization	L2-normalize every embedding before storage so cosine similarity = dot product, which is faster to index

CLIP vs. SigLIP MVP decision point

Aspect	CLIP	SigLIP
Training objective	Contrastive softmax over a batch (InfoNCE-style)	Sigmoid pairwise loss — does not require large batches to work well
Typical retrieval quality	Strong, extremely well documented baseline	Often slightly stronger on retrieval benchmarks at comparable size
Ecosystem maturity	Most tutorials, most prior art, easiest to debug against reference numbers	Newer, growing ecosystem, slightly fewer reference implementations
Recommendation	Start with CLIP for the baseline (best documented), then add a SigLIP run as an experiment (Section 16) to compare — this comparison itself is a strong portfolio artifact.	

Why NOT train a vision-language model from scratch

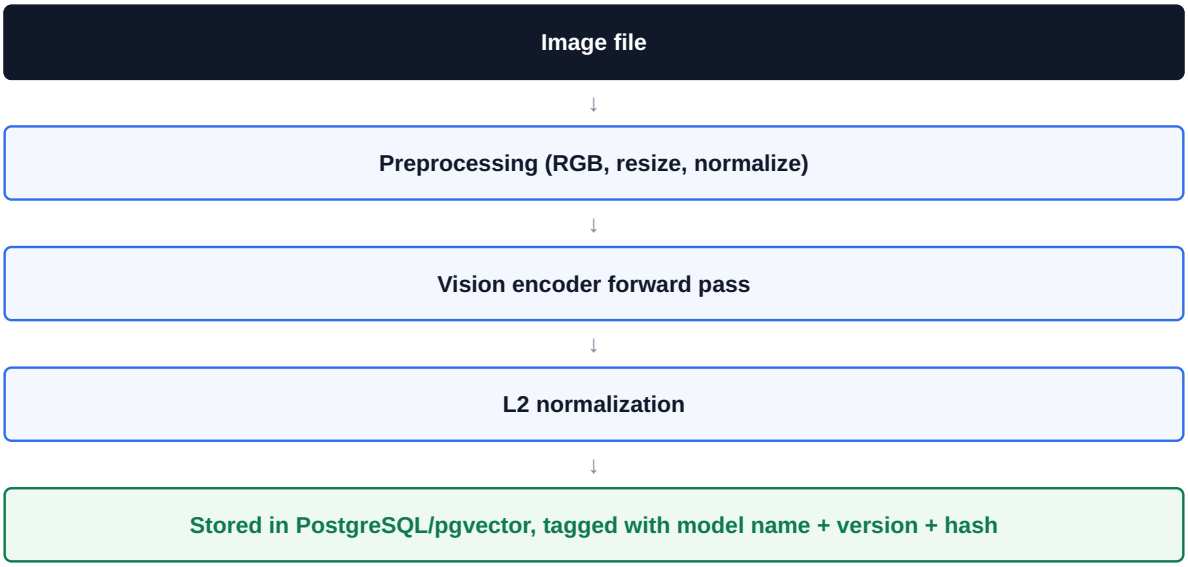
- Contrastive vision-language pretraining requires hundreds of millions of image-text pairs and thousands of GPU-hours — completely out of reach for a student project.
- The marginal engineering skill gained from attempting this does not match the cost; the valuable skill is knowing *when* to use a pretrained model versus training one, which is itself a core ML engineering judgment call.
- Fine-tuning (adapting an existing pretrained encoder on a small amount of domain data) is feasible and valuable — this is covered separately in Section 17, and is a fundamentally different, much cheaper operation than pretraining from scratch.

Advanced improvements ADVANCED

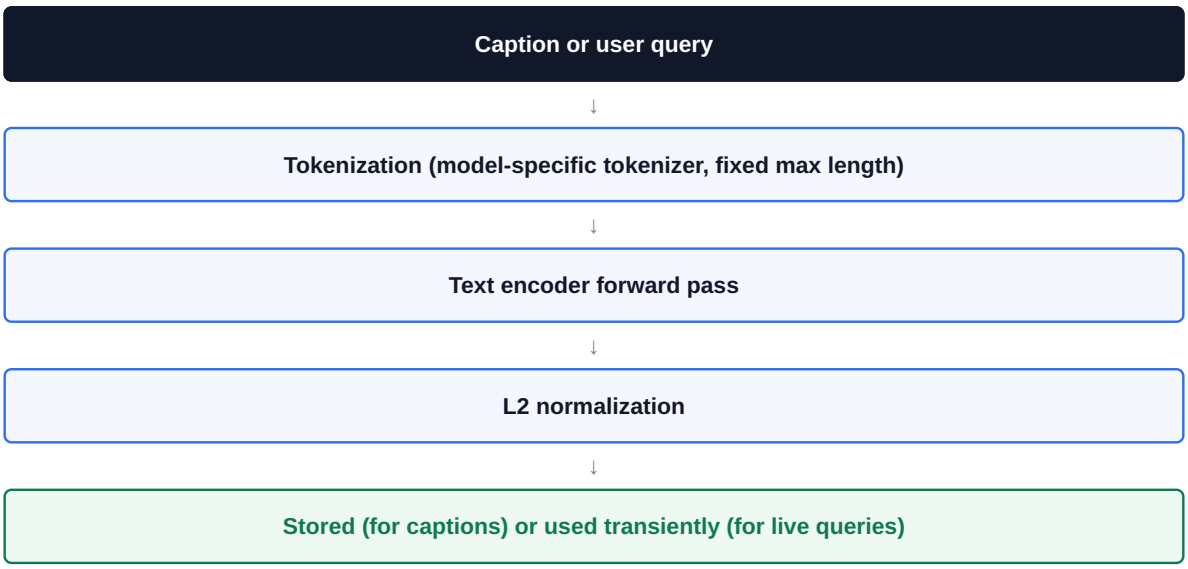
- Model comparison across multiple checkpoint sizes (ViT-B/32 vs ViT-L/14) to quantify the latency/quality tradeoff.
- Contrastive fine-tuning on domain-specific pairs (e.g. product images once on ABO).
- Hard-negative mining — sampling near-miss non-matching pairs during fine-tuning to sharpen the decision boundary.
- Domain-specific continued pretraining if a large enough in-domain corpus becomes available.
- A learned re-ranking model on top of retrieval candidates (Section 18).

10 Embedding Generation Pipeline

Image embedding pipeline



Text embedding pipeline



Batch processing MVP

Offline embedding generation (for the ~31k dataset images and ~159k captions) always runs in batches, not one item at a time — this is a 10–50x throughput difference on both CPU and GPU due to better hardware utilization and reduced per-call overhead. A batch size should be chosen based on available memory and tuned empirically (start around 32–64 for CPU, 128–256 for GPU).

GPU vs. CPU

Phase	Recommendation	Reasoning

One-time offline embedding of the full dataset	GPU if available (Colab/Kaggle free tier is enough for Flickr30k), else CPU with batching + patience	~31k images is a few hours on CPU, minutes on a free-tier GPU — a one-time cost either way
Live query embedding (single image or text at request time)	CPU is acceptable	Single-item inference latency on a small ViT-B/32 is in the tens-to-low-hundreds of milliseconds on CPU — fine for the latency targets in Section 4

Avoiding recomputation MVP

Every stored embedding is tagged with the model name, model version/checkpoint hash, and preprocessing config hash. Before regenerating an embedding, the pipeline checks whether a matching (image_id, model_version) pair already exists and skips it if so. This makes adding new images to an existing index cheap, and makes re-embedding the whole dataset after a model change an explicit, deliberate operation rather than an accidental one.

11 Database Design (PostgreSQL + pgvector)

Core tables

Table	Key fields	Purpose
images	image_id (PK), file_path/url, width, height, file_hash, split, ingested_at	One row per physical image, dataset-agnostic
captions	caption_id (PK), image_id (FK), text, language, created_at	One row per caption; supports the 5-captions-per-image structure of Flickr30k and generalizes to titles/descriptions for ABO
image_embeddings	embedding_id (PK), image_id (FK), model_name, model_version, embedding (vector type), created_at	Stores one vector per (image, model) pair — allows multiple model versions to coexist during evaluation
text_embeddings	embedding_id (PK), caption_id (FK, nullable), raw_text, model_name, model_version, embedding (vector type), created_at	Stores caption embeddings; live user queries are embedded on the fly and not necessarily persisted (optional, for caching)
models	model_name, model_version, embedding_dim, framework, source_checkpoint, registered_at	Registry of which encoder versions exist, so the API can report /model/info accurately
search_logs RECOMMENDED	request_id, query_type, query_text, latency_ms, top_k, model_version, timestamp	Feeds monitoring dashboards and future click-through-based evaluation
evaluation_runs RECOMMENDED	run_id, model_version, metric_name, metric_value, dataset_split, run_at, mlflow_run_id	Structured record of evaluation results, cross-referenced with MLflow

pgvector essentials

Concept	Explanation
Vector datatype	pgvector adds a native *vector(N)* column type storing a fixed-length float array directly in a Postgres row
Cosine distance	pgvector's cosine-distance operator returns *1 − cosine similarity*; since embeddings are pre-normalized, this is equivalent to ranking by dot product but keeps the standard "distance, smaller is better" convention pgvector's operators expect
Indexing	Without an index, pgvector does an exact brute-force scan — fine up to tens of thousands of rows, too slow beyond that for interactive latency
HNSW index	Graph-based approximate index; higher build cost and memory, but the best query latency/recall tradeoff for read-heavy workloads — **recommended default** once the index exceeds ~10k–20k rows
IVFFlat index	Cluster-based approximate index; cheaper to build, needs a good choice of cluster count and a representative sample before building, and its recall degrades more under data updates than HNSW's

When to use each	Flickr30k scale (31k images): HNSW is affordable to build and gives the best query-time recall/latency; IVFFlat is a reasonable fallback if build memory is constrained

Design notes

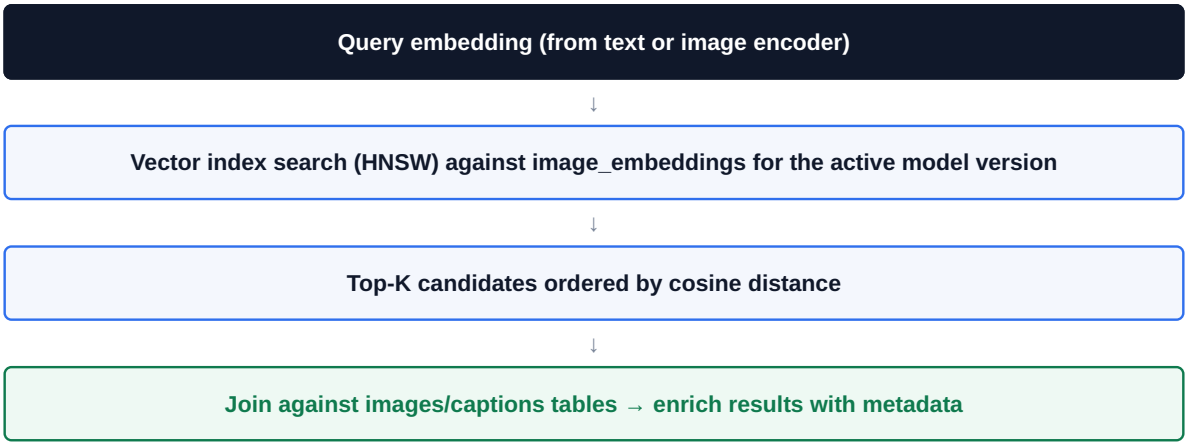
- Every embeddings table stores *model_name* + *model_version* alongside the vector, never just the vector alone — this is what makes A/B comparisons between CLIP and SigLIP possible without overwriting data.
- Foreign keys enforce that an embedding cannot exist for an image_id that was never ingested — this catches pipeline bugs at the database level, not just in application code.
- Timestamps on every table support debugging ("was this embedding generated before or after the last model swap?") and monitoring.

12 Retrieval System

Exact vs. approximate nearest-neighbor search

	Exact NN	Approximate NN (ANN)
Guarantee	Always finds the true closest vectors	Finds vectors that are very likely close, with a small, tunable chance of missing the true top result
Cost	Scales linearly with dataset size — fine at 31k vectors, painful past ~500k on modest hardware	Sub-linear query time via graph or cluster structures (HNSW/IVFFlat)
Use in this project	Acceptable for the initial MVP at Flickr30k scale, and useful as a "ground truth" reference when validating that the ANN index isn't losing meaningful recall	The production default once an index is built (Section 11)

Retrieval flow (Top-K candidate generation)



Retrieval vs. ranking vs. recommendation — a distinction worth stating clearly in interviews

Term	Definition in this system
Retrieval	Cheaply narrowing millions of items down to a small candidate set (here: Top-K from the vector index) — optimized for speed and recall, not perfect precision
Ranking	Taking that candidate set and ordering it as precisely as possible by relevance — can use a more expensive model (Section 18) since it only runs on a handful of candidates
Recommendation	A broader task that may use retrieval+ranking as a building block, but also incorporates signals this project does not model in its MVP: user history, collaborative filtering, business rules

13 Multimodal Search Strategy

Approach	How it works	Pros	Cons
A. Weighted embedding combination	combined vector = alpha × image_embedding + (1 − alpha) × text_embedding, then re-normalized and searched as a single query vector	Simple, one index lookup, fast, easy to reason about and tune	Naive averaging can wash out one modality if alpha is poorly chosen; assumes the two embeddings are directly combinable, which holds reasonably well for CLIP/SigLIP's shared space but is an approximation
B. Separate retrieval + score fusion	Run image-based and text-based retrieval independently, then merge/re-rank candidate lists by a weighted sum or reciprocal-rank fusion of their scores	Each modality's retrieval is untouched and independently debuggable; more robust when one modality's query is much stronger signal than the other	Two index lookups per request (roughly 2x retrieval cost); fusion weight still needs tuning
C. Dedicated multimodal fusion model	A small trained model takes both embeddings (or raw inputs) and outputs a single fused representation or relevance score	Can learn a non-linear, data-driven fusion that outperforms manual weighting	Requires labeled multimodal query data to train — not available for Flickr30k out of the box; real engineering and evaluation overhead

RECOMMENDATION FOR THE MVP

Use **Approach A (weighted embedding combination)** for the initial multimodal endpoint. It requires no additional training data, reuses the existing single-vector index and search code path unchanged, and is fast enough to ship as part of the MVP timeline. Expose `alpha` as a request parameter (default 0.5) so it can be evaluated and tuned empirically. Approach B is a reasonable RECOMMENDED upgrade once Approach A is evaluated and its weaknesses are visible in real query logs. Approach C is ADVANCED and only worth pursuing once there is a real evaluation set of multimodal queries to train and validate it against.

14 Evaluation Dataset Construction

Flickr30k's 5-captions-per-image structure gives a ready-made way to build a text→image evaluation set without any additional labeling: each caption is a natural-language query, and its own source image is the ground-truth positive.

Construction procedure

- **Query:** one held-out caption from the test split.
- **Positive image:** the image that caption was written for.
- **Negative images:** implicitly, every other image in the test split's index — retrieval metrics like Recall@K naturally handle this without needing explicitly labeled negatives.
- Only captions from the **test split** (Section 5's Karpathy split) are used to build evaluation queries — never captions the model's index was built from during development iteration.

TRAIN/TEST LEAKAGE RISKS

- If the vector index used for evaluation includes images whose captions leaked into any tuning decision (e.g. you eyeballed test-set results while choosing preprocessing hyperparameters), your Recall@K numbers are optimistic and not trustworthy.
- If duplicate or near-duplicate images (Section 6) span the train/val/test boundary, a query can trivially retrieve its near-duplicate rather than demonstrating real generalization — this is why duplicate detection during ingestion matters for evaluation integrity, not just storage efficiency.
- Because CLIP/SigLIP are pretrained (not trained by you on Flickr30k), classical "train/test leakage" mostly reduces to: don't let test-set observations influence your preprocessing or fusion-weight choices, and always report a validation-set intermediate result before touching the test set.

15 Evaluation Metrics

Metric	Meaning	Practical read
Recall@1	Fraction of queries where the true positive image is the #1 ranked result	The strictest, most interview-friendly headline number
Recall@5	Fraction of queries where the true positive appears in the top 5	Reflects a realistic "first screen of results" experience
Recall@10	Fraction of queries where the true positive appears in the top 10	A looser sanity metric — a system with poor Recall@10 has a broken embedding or index, not just imperfect ranking
Precision@K	Of the K results returned, the fraction that are relevant	Most meaningful when multiple ground-truth positives exist per query (e.g. any of the 5 captions' images counted as relevant for a broader notion of "relevant")
MRR (Mean Reciprocal Rank)	Average of 1/rank of the first correct result across all queries	Rewards getting the right answer *early*; heavily penalizes a correct-but-buried result
NDCG RECOMMENDED	Discounted cumulative gain normalized against the ideal ranking, supports graded (non-binary) relevance	Most useful once relevance is graded rather than binary — e.g. once re-ranking or ABO's structured attributes let you score "partially relevant" results

EXAMPLE CALCULATION — RECALL@5 AND MRR

Suppose 4 evaluation queries have their true positive image ranked at positions 1, 3, 12, and 2 respectively. Recall@5 counts how many of those ranks are ≤ 5: three of the four (ranks 1, 3, 2), giving Recall@5 = 3/4 = 0.75. MRR averages the reciprocal ranks: (1/1 + 1/3 + 1/12 + 1/2) / 4 ≈ (1 + 0.333 + 0.083 + 0.5) / 4 ≈ 0.479. Note how the query ranked 12th barely moves Recall@5 (it simply doesn't count) but still drags MRR down — this is why reporting both metrics together gives a fuller picture than either alone.

Target benchmark goals for the MVP

Metric	Realistic target with a pretrained CLIP ViT-B/32 baseline on Flickr30k test
Recall@1 (text→image)	≈ 25–35%
Recall@5 (text→image)	≈ 50–65%
Recall@10 (text→image)	≈ 65–78%

These are approximate ranges consistent with published zero-shot CLIP retrieval results on Flickr30k; treat them as a sanity-check band, not a hard requirement — the goal is a correctly implemented, honestly measured system, not chasing a specific number.

16 Baseline Experiments

#	Experiment	Purpose	Priority
1	Random baseline	Returns K random images regardless of query — establishes the floor every real approach must clear; expected Recall@K ≈ K/(index size)	MVP
2	CLIP baseline (ViT-B/32)	The primary MVP baseline — establishes the number everything else is compared against	MVP
3	SigLIP baseline	Direct comparison against CLIP under identical evaluation code	RECOMMENDED
4	Similarity metric comparison (cosine vs. dot product vs. Euclidean)	Confirms the metric choice interacts correctly with normalization; with L2-normalized vectors these should be equivalent for cosine/dot — a mismatch here usually reveals a normalization bug	RECOMMENDED
5	Embedding normalization strategies (raw vs. L2-normalized)	Quantifies why normalization matters for this system, not just asserts it	RECOMMENDED
6	Re-ranking on top of the best retrieval baseline	Measures the marginal value of the two-stage architecture from Section 18	ADVANCED

Experiment tracking table (log every run like this)

Run ID	Model	Index type	Recall@1	Recall@5	Recall@10	MRR	P50 latency
exp-001	random	—	~0.03%	~0.16%	~0.32%	—	—
exp-002	CLIP ViT-B/32	brute-force	fill in	fill in	fill in	fill in	fill in
exp-003	CLIP ViT-B/32	HNSW	fill in	fill in	fill in	fill in	fill in
exp-004	SigLIP base	HNSW	fill in	fill in	fill in	fill in	fill in

Every row in this table should correspond to a logged MLflow run (Section 23) with parameters and metrics recorded automatically, not filled in by hand after the fact.

17 Model Improvement Strategy

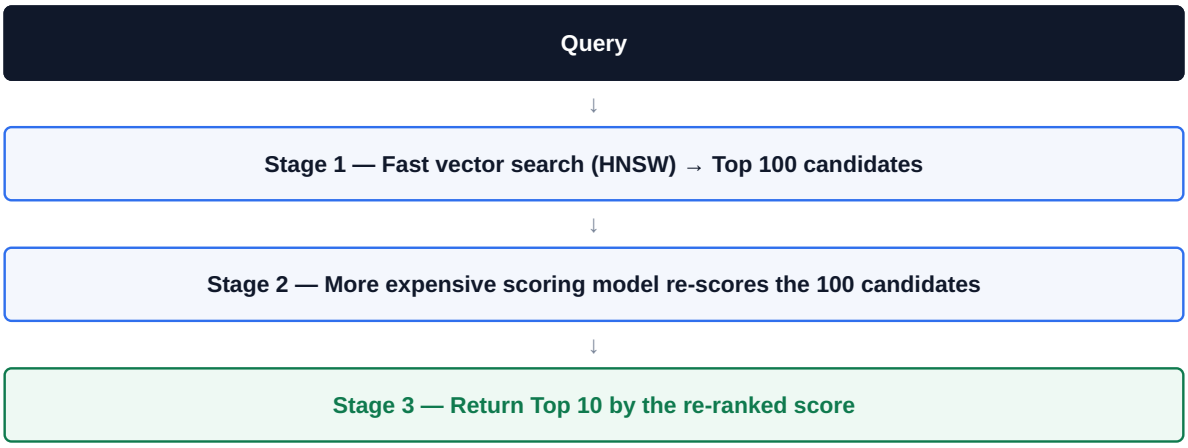
Technique	What it does	Cost/complexity	When it's worth it
Fine-tuning	Continue training the pretrained encoder's weights on your own image-text pairs	Medium — needs a training loop, GPU time, and careful learning-rate choice to avoid catastrophic forgetting	When baseline evaluation shows a systematic domain gap the pretrained model can't close zero-shot (e.g. once moved to ABO product images)
Contrastive loss (InfoNCE)	The standard loss for pulling matching pairs together and pushing non-matching pairs apart within a batch	Medium	Default choice if fine-tuning at all
Triplet loss	Explicit anchor/positive/negative triplets instead of in-batch negatives	Medium-high — needs a triplet mining strategy	When you have a strong reason to control exactly which negatives are used, rather than relying on random in-batch negatives
Hard negative mining	Deliberately sampling negatives that are close-but-wrong, rather than random negatives	Higher — needs a retrieval pass to find hard negatives before/during training	When easy random negatives stop improving the model further (diminishing returns visible in evaluation)
Data augmentation	Random crops/flips/color jitter on images during fine-tuning	Low-medium	Standard practice whenever fine-tuning with a small dataset, to reduce overfitting
Domain-specific data	Curating/expanding a dataset that matches your real target domain	Depends on data availability	Always more valuable than architecture changes if the current data doesn't represent the deployment domain
Better encoders	Swapping to a larger checkpoint (ViT-L/14 etc.)	Low code change, higher inference cost	When evaluation shows a genuine accuracy ceiling with the smaller model and latency budget allows it
Re-ranking	See Section 18	Medium	When retrieval Recall@10 is good but Recall@1/precision at the top is not, i.e. the right items are in the candidate set but not ranked first

WHEN FINE-TUNING IS ACTUALLY WORTH DOING

Fine-tune only after the pretrained baseline has been evaluated and a specific, measured gap has been identified — for example, zero-shot CLIP performing noticeably worse on a product-image domain than on natural photos. Fine-tuning without this evidence risks spending significant time and compute chasing an improvement that a cheaper change (better preprocessing, re-ranking, a bigger pretrained checkpoint) would have delivered for less cost. For the Flickr30k prototype specifically, fine-tuning is ADVANCED and not required — the pretrained baseline is strong enough on this general-domain data that the return on

> investment is low; fine-tuning becomes genuinely worthwhile once the project moves to a specialized domain like ABO product images (Section 31).

18 Two-Stage Re-Ranking ADVANCED



Why this two-stage architecture matters in production

A model expensive enough to give the best possible relevance judgment (a larger encoder, a cross-attention model that jointly looks at the query and each candidate, or a fusion of multiple signals) is almost always too slow to run against every item in a 30k–30M item index for every query. The two-stage pattern is the standard production answer: a cheap, highly scalable retrieval stage (Stage 1) narrows the space by orders of magnitude, and an expensive, higher-quality model (Stage 2) only has to score a small, fixed-size candidate set — so its cost stops depending on the size of the full index. This is the same pattern used by virtually every large-scale search and recommendation system in industry, which makes it a strong topic to speak to in interviews even in a scaled-down student implementation.

What Stage 2 can be, in increasing order of complexity

- **Simplest:** re-score the same candidates with a larger/more accurate encoder checkpoint than the one used for Stage 1's index.
- **Fusion re-ranking:** combine the Stage 1 similarity score with metadata signals (caption keyword overlap, recency, popularity) into a weighted score.
- **Cross-encoder:** a model that takes the (query, candidate) pair jointly as input rather than encoding them independently — typically the highest quality, and the most expensive per-candidate, which is exactly why it only runs on the small Stage-1 candidate set.

19 API Design

Folder structure (application layer)

Path	Purpose
app/main.py	FastAPI app instantiation, router registration, startup/shutdown hooks (load model once, open DB pool)
app/api/routes/	One module per resource: search.py, health.py, models.py
app/schemas/	Pydantic request/response models — the API's typed contract
app/services/	Business logic: embedding service, retrieval service, fusion service — kept independent of FastAPI so it's unit-testable without spinning up a server
app/db/	SQLAlchemy models/session management, pgvector query helpers
app/core/config.py	Environment-driven settings (DB URL, model name, thresholds) — no hardcoded values
app/core/logging.py	Structured logging setup

Endpoint contracts

Endpoint	Success response	Error handling
POST /search/text	200 with ranked results array, model_version, latency_ms	422 on empty/too-long query (Pydantic validation); 500 with a generic error body on unexpected failures, never a raw stack trace
POST /search/image	200 with ranked results array	413 if upload exceeds size limit; 415 if content-type isn't an accepted image format; 422 if the file can't be decoded as an image
POST /search/multimodal	200 with ranked results array, fusion alpha used	400 if neither image nor text is provided; same validation as the two endpoints above for whichever fields are present
GET /health	200 `{"status":"ok","db":"connected","model_loaded":true}`	503 if the database is unreachable or the model failed to load at startup
GET /model/info	200 with model_name, model_version, embedding_dim, index_type	—

Validation, error handling, and logging principles

- Every request body/query/file is validated by a typed schema before it reaches business logic — invalid input never silently propagates into the retrieval pipeline.

- Errors return a consistent JSON error shape (error code, human-readable message, request_id) rather than framework-default HTML error pages.
- Every request is logged with: endpoint, status code, latency, model_version, and a generated request_id that also appears in the response — this makes a specific user-reported issue traceable to specific log lines.

20 Frontend

The frontend's job is to make the ML system easy to demo and to inspect — it is deliberately kept simple so the majority of engineering effort stays on the retrieval system itself.

Feature	Behavior
Text search box	Debounced submit-on-enter, calls /search/text, shows a loading state while waiting
Image upload	Drag-and-drop or file picker, client-side preview before submit, calls /search/image
Multimodal search	Both inputs active at once with a visible fusion-weight slider (alpha) so the effect of Section 13's fusion approach is directly demoable
Result grid	Responsive grid of result cards: thumbnail, caption, similarity score, rank badge
Similarity scores	Shown as both a raw number and a simple visual bar, so non-technical viewers of the demo can interpret relative relevance at a glance
Loading state	Skeleton grid placeholders while a request is in flight
Errors	Human-readable inline message (e.g. "That file isn't a supported image format") instead of a blank screen or console-only error

A minimal React/Vite (or plain HTML/JS) single page is enough — no routing framework, no state management library, no design system needed. This matches the established stack preference for lightweight frontend tooling on portfolio projects.

21 Repository Structure

project/

Directory / file	Purpose
data/	Raw, processed, images, and metadata as defined in Section 5 — typically git-ignored except for small manifest/metadata files
notebooks/	EDA, experimentation, and one-off analysis — never imported by production code
src/	Installable Python package: data pipeline, embedding pipeline, retrieval logic, evaluation harness — the code notebooks and the API both import from here, so logic is written once
tests/	Mirrors src/ structure: unit, integration, and API test suites (Section 22)
configs/	YAML/JSON configuration files (model settings, index parameters, data paths) — loaded, not hardcoded
scripts/	Thin CLI entry points that call into src/ — e.g. run_ingestion, generate_embeddings, run_evaluation
models/	Local cache for downloaded checkpoints and any fine-tuned weights (git-ignored, tracked via MLflow/model registry instead)
docker/	Dockerfiles for the API service and any auxiliary services
frontend/	The search UI (Section 20)
docs/	Architecture diagram source, ADRs (architecture decision records), evaluation reports
.github/workflows/	CI/CD pipeline definitions (Section 24)
Dockerfile	Production image build for the API service
docker-compose.yml	Local multi-service orchestration: API + PostgreSQL/pgvector + (optionally) MLflow
requirements.txt / pyproject.toml	Pinned dependencies for reproducible environments
README.md	Project overview, architecture diagram, results, how to run — the single most-read file in the repo (Section 35)

22 Testing Strategy

Test type	What it covers	Concrete examples
Unit tests	Individual functions in isolation, no DB or network	Embedding normalization produces unit-length vectors; fusion function correctly weights alpha=0 (pure text) and alpha=1 (pure image); caption cleaning strips control characters correctly
Data tests	The data engineering pipeline's validation rules from Section 6	Corrupted-image fixture is correctly quarantined; a caption with 0 length is flagged; split assignment is deterministic given a fixed seed
Embedding tests	Encoder integration behaves as expected	Encoding the same image twice yields identical vectors (determinism); embedding dimension matches the registered model's expected dimension; a known matching image-caption pair scores higher than a random pair
Retrieval tests	Search logic against a small, controlled in-memory or test-database fixture	Querying with an embedding identical to a stored one returns that item as rank 1; Top-K respects the requested K; results are correctly sorted by score
Integration tests	Multiple components together, including a real (test) database	Full pipeline from raw query text → embedding → pgvector search → enriched result, run against a seeded test Postgres instance
API tests	The FastAPI layer itself, via its test client	POST /search/text with a valid query returns 200 and the expected response schema; an oversized image upload returns 413; /health reflects a simulated DB outage as 503

COVERAGE TARGET

Aim for >70% coverage concentrated on `src/` business logic (embedding, retrieval, fusion, data validation) rather than chasing 100% coverage on trivial glue code. CI (Section 24) runs the full suite on every pull request and blocks merges on failure.

23 MLOps: Experiment Tracking, Versioning, Reproducibility

MLflow usage

MLflow component	What is logged
Experiments	One MLflow experiment per project phase — e.g. "baseline-retrieval", "reranking-experiments"
Parameters	Model name/checkpoint, index type (HNSW/IVFFlat/brute-force), K, fusion alpha, batch size, preprocessing config hash
Metrics	Recall@1/5/10, Precision@K, MRR, NDCG, P50/P95 latency — logged per evaluation run
Artifacts	Evaluation report (plots/tables), the exact evaluation-query manifest used, confusion cases (worst-performing queries) for manual inspection
Model versions	Registered checkpoint reference (Hugging Face model ID + revision hash, or fine-tuned weights path) tied to each experiment run

Model versioning

Pretrained checkpoints are referenced by an exact, pinned identifier (model name + revision/commit hash from the Hugging Face Hub), never a floating "latest" tag — this guarantees that re-running the pipeline six months later reproduces the same embeddings. Any fine-tuned model gets a semantic version tag (e.g. `clip-flickr30k-v1`) registered in both MLflow's model registry and the project's own `models` database table.

Data versioning

The dataset version manifest from Section 6 (row counts, file hashes, pipeline code commit) is itself an artifact — logged alongside each MLflow run so that every reported metric can be traced back to the exact data snapshot that produced it. For a project this size, a lightweight manifest-file approach is sufficient; a dedicated data-versioning tool (DVC) is optional and only worth adding if the dataset starts changing frequently.

Reproducibility checklist

- Fixed random seeds everywhere sampling/splitting occurs, and the seed value itself is logged.
- Pinned dependency versions (requirements.txt/pyproject.toml with exact versions, not ranges).
- Pinned model checkpoint revisions, never "latest."
- Every evaluation run's git commit hash logged in MLflow, so results can be tied back to the exact code that produced them.

24 CI/CD

Workflow	Trigger	Steps
lint-and-test	Every push and pull request	Install dependencies → run linter/formatter check → run unit + data + retrieval tests → upload coverage report
integration-test	Pull requests targeting main	Spin up a throwaway PostgreSQL/pgvector service container → run integration and API test suites against it
docker-build	Pull requests and pushes to main	Build the API Docker image → run a smoke test (container starts, /health returns 200) → fail fast if the image is broken before it's ever deployed
deploy	Merge to main (or a manual approval gate)	Build and push the image to a registry → deploy to the hosting target from Section 27 → run a post-deploy /health check

Design principles for the pipeline

- Tests run before the Docker build, and the Docker build runs before deploy — each stage is a hard gate for the next, so a broken change never reaches production.
- Secrets (DB credentials, registry tokens) are stored as CI secrets, never committed, and injected as environment variables at build/deploy time.
- The deploy step is the only one with write access to production infrastructure — PR-triggered workflows are read-only/test-only.

25 Monitoring & Observability

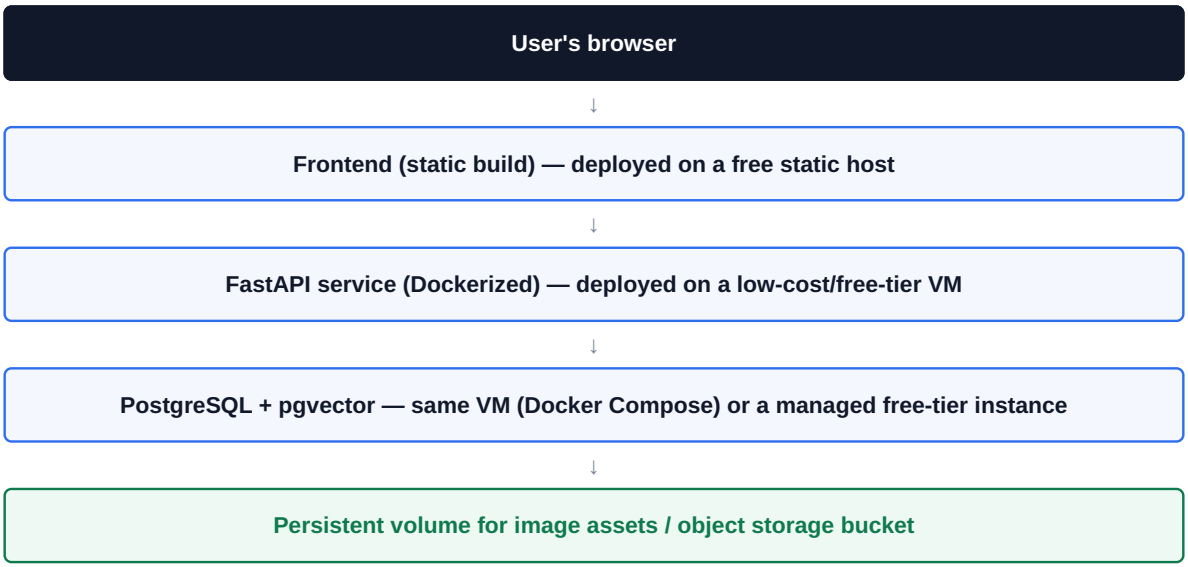
Category	Metrics to log	Priority
API performance	Request latency (P50/P95/P99) per endpoint, throughput (requests/sec), error rate by status code	MVP
Model serving	Embedding generation latency (separately for image vs. text), currently loaded model_version	MVP
Retrieval performance	Vector search latency, average/percentile similarity score of top result (a proxy signal — a sudden drop across many queries can indicate an index or model problem)	RECOMMENDED
Data drift ADVANCED	Distribution shift in incoming query embeddings vs. the indexed corpus (e.g. PSI on embedding-space statistics) — signals when live traffic looks meaningfully different from what the system was validated on	ADVANCED
Embedding drift ADVANCED	Detects if a silent model or preprocessing change altered embeddings without a version bump	ADVANCED
Retrieval quality over time ADVANCED	If click-through or user feedback signals are ever captured, track realized relevance over time, not just offline Recall@K	ADVANCED

For the MVP, structured request logs (Section 19) plus a simple metrics endpoint scraped by Prometheus and visualized in a small Grafana dashboard (latency, error rate, request volume, model version) is enough to demonstrate real observability practice without building a full ML-monitoring platform.

26 Security

Concern	Mitigation
File upload validation	Whitelist accepted MIME types and file extensions; verify the actual file content matches the declared type, not just the extension
Image size limits	Hard cap on upload size (e.g. 10MB) and on decoded pixel dimensions, enforced before the file is passed to any decoding library
Malicious file handling	Decode uploads in a way that fails safely on malformed/crafted files (image decompression bombs, corrupted headers) rather than crashing the process or consuming unbounded memory
API rate limiting	Per-IP or per-API-key request rate limits on the search endpoints to prevent abuse of a public demo
Input validation	Every field validated by a typed schema (Section 19) before touching business logic
Secrets management	Database credentials and any API keys read from environment variables, never committed; local development uses a git-ignored .env file with an .env.example template committed instead
Database credentials	A least-privilege application DB user (read/write only on the tables it needs), separate from any admin/migration user
CORS	Restrict allowed origins to the known frontend domain(s) in production; permissive only in local development
Authentication	Not required for a public read-only demo; if the project grows to support saved searches or personalization, add token-based auth at that point — explicitly out of scope for the MVP

27 Deployment Architecture



Layer	Realistic low-cost option	Notes
Backend compute	Oracle Cloud Always Free ARM VM (already the established preference), or Fly.io/Render free tier	Enough CPU for CPU-only inference at this dataset scale
PostgreSQL + pgvector	Self-hosted via Docker Compose on the same VM	Simplest to operate for a single-instance student deployment; avoids managed-DB costs
Frontend hosting	Static hosting (e.g. a free static-site host) serving the built frontend bundle	Decouples frontend deploys from backend deploys
Image asset storage	Local disk/volume for the Flickr30k-scale dataset (a few GB fits comfortably); object storage only becomes necessary at much larger scale	Keeps the architecture simple at this size
Environment variables	Injected via the hosting platform's secret/env configuration, matching local .env.example	Same config-driven approach as local development

CPU vs. GPU deployment

Serve on CPU. As established in Section 10, single-query embedding latency for a ViT-B/32-scale encoder is well within the project's latency targets on CPU, and GPU hosting is both significantly more expensive and unnecessary at this traffic scale. GPU is only used transiently (a free-tier Colab/Kaggle session) for the one-time offline embedding of the full dataset.

Minimizing infrastructure cost

- Single always-free VM running the API, database, and reverse proxy via Docker Compose — no separate managed services to pay for.
- Model checkpoint downloaded once and cached in the Docker image or a persistent volume, not re-downloaded on every container restart.
- Precomputed embeddings for the entire indexed dataset mean production inference only ever runs on the live query (one image or one text string), not the whole corpus.

28 Performance Optimization

Technique	Applies to	Impact
Batching	Offline embedding generation	10–50x throughput vs. one-at-a-time inference (Section 10)
Caching	Repeated identical text queries	Skips re-encoding for common queries; a small in-process LRU cache keyed on the normalized query string is enough at this scale
Precomputed embeddings	All indexed images/ captions	The single biggest latency win — live requests never re-embed the corpus, only the query
Vector indexes (HNSW)	pgvector search	Sub-linear query time vs. brute-force scan, essential once the index passes roughly 10k–20k rows
Approximate nearest-neighbor search	Same as above	Trades a small, tunable recall loss for large latency gains
Quantization ADVANCED	Stored embeddings or model weights	Reduces memory footprint and can speed up inference; only worth the added complexity if memory/latency profiling shows it's actually the bottleneck
Model size choice	Encoder selection	ViT-B/32 over ViT-L/14 unless evaluation shows a meaningful quality gap that justifies the extra latency
CPU optimization	Serving	Use the framework's inference-mode/no-grad context, avoid unnecessary tensor copies, keep the model loaded once at process startup rather than per-request
GPU inference ADVANCED	High-traffic scenarios only	Not needed at this project's traffic scale; documented here as the next lever if the deployed demo ever needed to handle significant concurrent load

Latency benchmarks to actually measure and report

Measurement	Target
Text query embedding time	< 50ms CPU
Image query embedding time (incl. decode+preprocess)	< 150ms CPU
pgvector HNSW search (31k index)	< 20ms
End-to-end /search/text P50	< 300ms
End-to-end /search/image P50	< 500ms

29 Failure Cases & Diagnostics

Failure mode	Likely cause	Diagnosis	Fix
Irrelevant results	Normalization mismatch, wrong preprocessing config, or a genuinely hard/ambiguous query	Check the raw similarity scores — unusually low top-1 scores across many queries point to a systemic bug, not query difficulty	Verify preprocessing matches the encoder's documented recipe (Section 8); re-check normalization (Section 10)
Duplicate results	Near-duplicate images in the index (Section 6)	Inspect result sets for images sharing a hash/perceptual-hash cluster	Deduplicate at ingestion, or add a diversity/de-dup pass to the ranking layer
Poor captions surfaced	Some Flickr30k captions are generic or low-quality	Manually review low-relevance-scoring returned captions	Prefer the caption with highest text-embedding similarity to the image among the 5 available, rather than always the first
Ambiguous queries	Query genuinely matches many different images equally well (e.g. "a person outside")	Check whether top results are diverse-but-plausible rather than wrong	Not a bug — document as an expected limitation; result diversity re-ranking can help but is optional
Unusual/out-of-distribution images	Uploaded image is very unlike anything in Flickr30k's domain	Compare uploaded image's embedding norm/neighborhood density to typical indexed images	Return results with visibly low similarity scores rather than hiding the mismatch — let the score communicate low confidence
Corrupted files	Malformed upload or corrupted dataset file slipping past validation	Reproduce with the exact file, check decode step in isolation	Tighten validation (Section 6/26); return a clear 422/415 rather than a 500
Model failures	Checkpoint download failure, version mismatch, OOM on batch encode	Check /health and startup logs for model-load errors	Pin checkpoint revision, add startup retries, cap batch size to available memory
Database failures	Connection pool exhaustion, index not built, migration not applied	Check /health's DB status field; check pgvector extension and index existence	Connection pooling with sane limits; startup check that required extensions/indexes exist
High latency	Missing/degraded index, cold model on first request, oversized uploads	Compare logged per-stage latencies (embed vs. search vs. serialize) from Section 25's logs	Warm the model at startup, confirm HNSW index is in use (not a fallback sequential scan), enforce upload size limits

30 Advanced Version Roadmap ADVANCED

Once the MVP is deployed and evaluated, these are the highest-leverage next steps, roughly in the order they'd naturally get tackled:

1. **Product dataset (ABO)** — swap in a domain with real business relevance (Section 31).
2. **Fine-tuned model** — contrastively fine-tune the encoder on the product domain once a measured gap justifies it (Section 17).
3. **User feedback capture** — log which results users click on, building the raw signal needed for anything beyond offline evaluation.
4. **Personalized recommendations** — incorporate per-user history into ranking, moving from pure retrieval toward true recommendation (Section 12's distinction).
5. **Click-through data** — once collected, use it to validate offline Recall@K/MRR against real user behavior, and to mine hard negatives for fine-tuning.
6. **Learning-to-Rank** — train a ranking model (e.g. LightGBM LambdaMART) on top of retrieval candidates using engineered features plus embedding similarity — a natural Stage 2 for the re-ranking architecture in Section 18.
7. **Hybrid search** — combine dense vector retrieval with sparse lexical search (e.g. BM25) so exact keyword/brand/SKU matches are never missed by a purely semantic system — a well-known weakness of pure embedding search.
8. **Text + image + metadata fusion** — extend the multimodal fusion strategy (Section 13) to also incorporate structured attributes (category, price range, brand) once available from a product dataset.
9. **Recommendation feedback loop** — periodically retrain/re-evaluate using accumulated feedback, closing the loop from monitoring (Section 25) back into model improvement.

31 Transition: Flickr30k → E-Commerce (ABO)

	Flickr30k	Amazon Berkeley Objects (ABO)
Visual content	Everyday photos, natural scenes	Studio product photography, often multiple angles per item
Text content	5 free-form captions per image	Product title, bullet-point description, and structured attributes (brand, color, material, category, dimensions)
Metadata richness	Minimal — essentially just captions	Rich structured metadata suitable for filtering and fusion, not just free-text search
Query intent	General scene description	Specific product attributes and purchase intent ("waterproof hiking boots size 42")
Evaluation ground truth	Caption ↔ image pairing (Section 14)	Can additionally use category/attribute matching, and eventually real click/purchase signals

What stays unchanged

- The database schema (Section 11) — `captions` becomes the natural home for product titles/descriptions with almost no structural change; a new `attributes` table (or a JSONB column) extends `images` for structured fields.
- The embedding pipeline (Section 10) — same encoder, same preprocessing philosophy, same storage-and-reuse logic.
- The pgvector indexing and retrieval logic (Sections 11–12) — dataset-agnostic by construction.
- The FastAPI service, its endpoints, and its testing/CI/CD/deployment setup (Sections 19, 22–27) — none of this is Flickr30k-specific.
- The evaluation metric definitions (Section 15) — Recall@K/MRR/NDCG apply identically; only the ground-truth construction changes.

What changes

- The data ingestion pipeline's parsing logic (Section 6) — ABO's metadata format differs from Flickr30k's caption file and needs its own ingestion adapter, not a rewrite of the pipeline's structure.
- The evaluation-set construction (Section 14) — ground truth shifts from caption-authorship pairing to category/attribute-based relevance, and eventually to real interaction data.
- The multimodal fusion inputs (Section 13) — structured attributes become a third signal alongside image and text embeddings.
- Licensing — ABO has different licensing terms than Flickr30k research data; verify current terms before any commercial use.
- Whether fine-tuning is worth it (Section 17) — a specialized product-image domain is exactly the scenario where fine-tuning the pretrained encoder is most likely to show a measurable, worthwhile improvement.

WHY THIS TRANSITION STORY MATTERS FOR A PORTFOLIO

Being able to say "I designed the schema and pipeline to be dataset-agnostic, validated it end-to-end on Flickr30k, and can point to exactly which four things would change to run it on a product catalog"

> demonstrates system design thinking, not just the ability to fine-tune a model. This is precisely the kind of judgment interviewers probe for at the ML engineer level.

32 Project Milestones

Milestone 1 — Dataset **Difficulty: Low**

Objective: A validated, versioned Flickr30k dataset ready for embedding.

Tasks: Download raw data; implement integrity, corruption, duplicate, and caption-quality checks; produce the Karpathy-style train/val/test split; write the EDA report.

Expected output: `data/processed/` populated, plus a data-quality report in `data/metadata/`.

Acceptance criteria: Zero corrupted/undecoded images remain in the processed set; split assignment is reproducible from a fixed seed; EDA report covers every plot in Section 7.

Dependencies: None.

Milestone 2 — Database **Difficulty: Low–Medium**

Objective: PostgreSQL + pgvector running locally with the schema from Section 11.

Tasks: Stand up Postgres via Docker Compose; install pgvector; create tables (images, captions, image_embeddings, text_embeddings, models); write a seed/migration script.

Expected output: A running local database, schema applied via migrations, images/captions loaded from Milestone 1's output.

Acceptance criteria: Schema matches Section 11; foreign key constraints enforced; a smoke-test insert/query round-trips correctly.

Dependencies: Milestone 1.

Milestone 3 — Embedding pipeline **Difficulty: Medium**

Objective: Every image and caption embedded with CLIP and stored in pgvector.

Tasks: Implement batched image/text encoding (Section 10); write embeddings with model/version tags; verify no unnecessary recomputation on re-run.

Expected output: Fully populated image_embeddings and text_embeddings tables for the CLIP baseline.

Acceptance criteria: Every image has exactly one embedding per model_version; re-running the script does not duplicate or recompute existing rows.

Dependencies: Milestone 2.

Milestone 4 — Baseline retrieval **Difficulty: Medium**

Objective: Working text→image and image→image search against pgvector, without an API yet.

Tasks: Implement query embedding + pgvector search functions; build the HNSW index; validate results qualitatively on sample queries.

Expected output: A callable Python retrieval function returning Top-K results for a given query.

Acceptance criteria: A hand-picked query ("a dog on a beach") returns visibly relevant images in the top results.

Dependencies: Milestone 3.

Milestone 5 — Evaluation **Difficulty: Medium**

Objective: Quantified retrieval quality with tracked experiments.

Tasks: Build the evaluation query set (Section 14); implement Recall@K/MRR/NDCG (Section 15); run and log the random and CLIP baselines (Section 16) in MLflow.

Expected output: An MLflow experiment with at least two logged runs and a results table.

Acceptance criteria: CLIP baseline clears the random baseline by a wide margin on every metric; results are reproducible on re-run.

Dependencies: Milestone 4.

Milestone 6 — Model improvement **Difficulty: Medium (optional depth: High)**

Objective: At least one evaluated improvement over the raw CLIP baseline.

Tasks: Run the SigLIP comparison experiment; run the normalization/metric ablations from Section 16; document findings.

Expected output: An updated experiment tracking table and a short written comparison.

Acceptance criteria: At least 2 additional experiment rows logged beyond the baseline, with a clear conclusion about which configuration is used going forward.

Dependencies: Milestone 5.

Milestone 7 — API **Difficulty: Medium**

Objective: The retrieval logic exposed as a documented FastAPI service.

Tasks: Implement /search/text, /search/image, /health, /model/info (Section 19); add request validation and structured logging.

Expected output: A running local API with interactive docs (OpenAPI/Swagger).

Acceptance criteria: All MVP endpoints return correct, schema-valid responses; invalid input returns the correct error codes.

Dependencies: Milestone 4.

Milestone 8 — Frontend **Difficulty: Low–Medium**

Objective: A usable search UI against the running API.

Tasks: Build the search box, upload control, result grid, loading/error states (Section 20).

Expected output: A demoable local frontend.

Acceptance criteria: All three search modes (text, image, multimodal if implemented) work end-to-end from the browser.

Dependencies: Milestone 7.

Milestone 9 — Docker **Difficulty: Low–Medium**

Objective: The full stack runs with a single command.

Tasks: Write the API Dockerfile; write docker-compose.yml wiring API + Postgres/pgvector (+ frontend if containerized).

Expected output: `docker compose up` produces a working system from a clean machine.

Acceptance criteria: A fresh clone + docker compose up (with data volume mounted) reaches a working /health endpoint with no manual steps.

Dependencies: Milestones 7–8.

Milestone 10 — CI/CD **Difficulty: Medium**

Objective: Automated testing and build gating on every change.

Tasks: Write pytest suites (Section 22); write the GitHub Actions workflows (Section 24).

Expected output: Green CI badge on the repository; a Docker image builds successfully in CI.

Acceptance criteria: A deliberately broken test fails the pipeline and blocks merge; a passing PR builds and smoke-tests the Docker image.

Dependencies: Milestone 9.

Milestone 11 — Monitoring **Difficulty: Medium**

Objective: Basic production observability in place.

Tasks: Add structured request logging; expose a Prometheus metrics endpoint; build a minimal Grafana dashboard (Section 25).

Expected output: A dashboard showing live request latency, error rate, and volume.

Acceptance criteria: A deliberately induced error (e.g. stopping the DB) is visible on the dashboard within the scrape interval.

Dependencies: Milestone 9.

Milestone 12 — Deployment **Difficulty: Medium**

Objective: A live, publicly reachable demo.

Tasks: Provision the VM (Section 27); wire the CI/CD deploy step; deploy frontend to static hosting; verify end-to-end from a fresh browser session.

Expected output: A public URL serving the working application.

Acceptance criteria: All acceptance criteria from Milestones 1–11 hold true against the deployed instance, not just locally.

Dependencies: All prior milestones.

33 Week-by-Week Implementation Plan

Assumes part-time work (roughly 8–12 hours/week alongside studies), spread over 8 weeks. Adjust pacing, not scope, if a week runs short.

Week 1 — Data foundations

Learning objectives: Flickr30k structure, image integrity checks, reproducible splitting.

Coding tasks: Download and organize the dataset; implement integrity/corruption/duplicate checks; implement the split logic.

Experiments: None yet.

Deliverables: Populated `data/processed/`, a data-quality report.

Tests: Data validation unit tests.

Documentation: Short README note on dataset provenance and license.

Week 2 — EDA and database

Learning objectives: Retrieval-relevant EDA; SQL basics; pgvector fundamentals.

Coding tasks: Produce the EDA notebook/report (Section 7); stand up PostgreSQL + pgvector via Docker Compose; create the schema.

Experiments: None yet.

Deliverables: EDA report; running local database with applied schema.

Tests: Schema/migration smoke test.

Documentation: Schema diagram (Section 11) drafted in docs/.

Week 3 — Embedding pipeline

Learning objectives: CLIP architecture basics, batching, embedding normalization.

Coding tasks: Implement and run the batched image/text embedding pipeline against the full dataset; persist to pgvector.

Experiments: Sanity-check embedding determinism.

Deliverables: Fully populated embeddings tables.

Tests: Embedding determinism and dimension tests.

Documentation: Note on model checkpoint and pinned revision used.

Week 4 — Retrieval and evaluation

Learning objectives: HNSW indexing, information retrieval metrics.

Coding tasks: Build the HNSW index; implement text→image and image→image search; build the evaluation harness and metric implementations (Sections 14–15).

Experiments: Random baseline, CLIP baseline (Section 16, experiments 1–2), logged to MLflow.

Deliverables: First evaluation report with Recall@K/MRR numbers.

Tests: Retrieval correctness tests.

Documentation: Results table added to README (placeholder acceptable at this stage).

Week 5 — API

Learning objectives: FastAPI request/response modeling, error handling patterns.

Coding tasks: Implement /search/text, /search/image, /health, /model/info; add validation, logging, and error handling (Section 19).

Experiments: None new; optionally the SigLIP comparison (Section 16, experiment 3) if time allows in parallel.

Deliverables: Running local API with interactive docs.

Tests: API test suite (Section 22).

Documentation: API request/response examples in README.

Week 6 — Frontend and Docker

Learning objectives: Multi-service Docker Compose orchestration.

Coding tasks: Build the frontend (Section 20); write Dockerfile and docker-compose.yml (Section 21/ Milestone 9).

Experiments: None.

Deliverables: Full stack runnable with a single command.

Tests: Manual end-to-end pass through all search modes.

Documentation: "How to run locally" section in README.

Week 7 — CI/CD and monitoring

Learning objectives: GitHub Actions workflow design, Prometheus/Grafana basics.

Coding tasks: Write GitHub Actions workflows (Section 24); add structured logging and a metrics endpoint; build a basic Grafana dashboard (Section 25).

Experiments: Optional: normalization/metric ablations (Section 16, experiments 4–5) if time allows.

Deliverables: Green CI on the repository; a working dashboard.

Tests: CI failing/passing behavior verified deliberately.

Documentation: CI badge and dashboard screenshot added to README.

Week 8 — Deployment and portfolio polish

Learning objectives: Deploying a Dockerized multi-service app to a low-cost VM.

Coding tasks: Provision the VM, wire the deploy CI step, deploy frontend and backend, verify end-to-end publicly (Section 27).

Experiments: Final consolidated results table across all logged experiments.

Deliverables: Live public demo URL.

Tests: Full acceptance criteria from Milestone 12 verified against the live deployment.

Documentation: Final README pass (Section 35), architecture diagram finalized, demo GIF recorded.

34 Learning Requirements

Before this stage	Must know	Nice to know
Embeddings (Sections 9–10)	What a vector representation is; cosine similarity; basic Transformer/CNN intuition	The details of contrastive loss/ InfoNCE; Vision Transformer internals
pgvector / database (Section 11)	Basic SQL (SELECT/JOIN/WHERE); what an index is conceptually; what a foreign key enforces	Internals of HNSW graph construction; IVFFlat clustering theory
Evaluation (Sections 14–15)	What precision and recall mean; how ranking metrics differ from classification accuracy	NDCG's discounting formula derivation; statistical significance testing between runs
API design (Section 19)	HTTP methods and status codes; what request/response validation is for	Async request handling internals in FastAPI/ASGI
Deployment (Section 27)	Docker basics (image vs. container, Dockerfile, Compose); basic Linux (SSH, file permissions, processes); what a reverse proxy does	Kubernetes; managed cloud vector-database services
MLOps (Section 23)	Why experiment tracking matters; what "reproducibility" means in practice	Full data-versioning tools (DVC) beyond a manifest-file approach

35 Final Portfolio Presentation

Element	Guidance
Project title	"Multimodal Semantic Search Engine" — short, descriptive, no unnecessary jargon
One-line description	"A production-style image/text retrieval system built on CLIP embeddings, PostgreSQL+pgvector, and FastAPI — deployed end-to-end from a research dataset to a running API."
Technical description	2–3 short paragraphs covering: the retrieval architecture, the evaluation methodology, and the MLOps/ deployment layer — mirroring Sections 1, 15, and 23–27 of this plan
Architecture diagram	The end-to-end flow from Section 27, rendered as a clean image directly in the README (not just linked)
Screenshots to include	The search UI mid-results for a text query and for an image query; the Grafana dashboard; the MLflow experiment comparison view
Metrics to report	Recall@1/5/10, MRR, compared across at least two model configurations (Section 16), plus P50/P95 latency
Demo requirements	A live link if the deployment is kept running, or a short screen-recorded GIF if not — a static repo with no visible working demo is the single biggest portfolio weakness for this kind of project
README structure	Title → one-liner → demo link/GIF → architecture diagram → key results table → how to run locally (Docker Compose) → design decisions & tradeoffs → roadmap (Section 30)

EXAMPLE STRONG CV BULLET

"Built and deployed a multimodal semantic search engine (CLIP, PostgreSQL/pgvector, FastAPI, Docker) over 31k images, achieving Recall@5 of [X]% and P95 latency under 800ms; established a reproducible evaluation harness, MLflow experiment tracking, and CI/CD with automated testing, applying the same architecture to a validated migration path toward a product-search domain."

36 Interview Preparation

Question	Strong junior ML engineer answer
What is CLIP and how does it work?	A dual-encoder model with a separate image encoder and text encoder, trained contrastively so matching image-text pairs are pulled close together in a shared embedding space and non-matching pairs are pushed apart — enabling zero-shot cross-modal comparison via cosine similarity.
Why normalize embeddings before storing them?	L2 normalization makes cosine similarity equivalent to a dot product, which is cheaper to compute and lets the vector index use a simpler, faster distance function without changing the ranking.
What is a vector database, and why not just use NumPy/brute-force search?	A vector database (or an extension like pgvector) provides indexed approximate nearest-neighbor search that scales sub-linearly with data size, along with persistence, transactional guarantees, and the ability to join vector results against relational metadata — brute-force NumPy search is fine at small scale but doesn't scale or persist.
Explain cosine similarity vs. Euclidean distance for embeddings.	Cosine similarity measures the angle between vectors, ignoring magnitude — appropriate when the direction of the embedding carries the semantic meaning, which is the case for normalized CLIP-style embeddings; Euclidean distance is sensitive to magnitude and is a different notion of "closeness" that matters less once vectors are unit-normalized.
What is approximate nearest-neighbor (ANN) search, and why use it?	ANN search trades a small, bounded chance of missing the exact best match for large gains in query speed via structures like HNSW graphs or IVFFlat clusters — necessary once brute-force scanning becomes too slow for interactive latency at scale.
How does HNSW work, at a high level?	It builds a multi-layer graph where each node connects to its approximate nearest neighbors; search starts at a sparse top layer and greedily descends toward denser layers, quickly narrowing to a small neighborhood without scanning the whole dataset.
What does Recall@K measure, and why report multiple K values?	The fraction of queries whose true relevant item appears within the top K results; reporting several K values (1, 5, 10) shows both strict top-result quality and a more forgiving "is it on the first page" view, since a system can differ sharply between them.
What's the difference between MRR and Recall@K?	Recall@K is binary per query (hit or miss within K); MRR is continuous and rewards ranking the correct answer earlier even within a hit — so MRR is more sensitive to ranking quality among the results that are already retrieved correctly.
When would you fine-tune a pretrained model instead of using it zero-shot?	Once evaluation reveals a measurable, systematic gap on your target domain that zero-shot performance can't close — fine-tuning without that evidence risks spending compute on a problem a cheaper fix (better preprocessing, bigger checkpoint, re-ranking) might solve.
What is contrastive learning, briefly?	A training approach that pulls representations of "positive" (matching) pairs together and pushes "negative" (non-matching) pairs apart in embedding space, typically using an InfoNCE-style loss over in-batch negatives.
Why use a two-stage retrieve-then-rerank architecture?	A cheap, scalable retrieval stage narrows a huge index down to a small candidate set, so a more expensive, higher-quality ranking model only has to run on that small set — decoupling ranking quality from index size, which is how large-scale systems stay both accurate and fast.
	FastAPI's built-in typed request/response validation (Pydantic) and native async support are a natural fit for an ML serving layer with strict input contracts and I/O-bound work; Flask is

Why FastAPI over Flask for this kind of service, or vice versa?	simpler and perfectly adequate when async isn't a priority — the right choice depends on team familiarity and whether async I/O is actually needed, not a universal rule.
Why Docker for this project?	It guarantees the API, its dependencies, and the database run identically in development, CI, and production, eliminating "works on my machine" failures and making the deployment step a simple image pull/run rather than a manual environment setup.
How would this system scale beyond 31k images?	The retrieval architecture already uses an ANN index (HNSW) so query latency degrades gracefully; at much larger scale (millions+), the next levers are index sharding, moving to a dedicated vector database service, and caching hot queries — none of which require changing the API contract.
How do you monitor an ML system in production, beyond standard API metrics?	Track model-specific signals like embedding generation latency and model version alongside standard latency/error-rate metrics, and — at a more advanced stage — monitor for data or embedding drift that would silently degrade retrieval quality without triggering a conventional error.

37 Final A-to-Z Checklist

- [ ] Dataset downloaded, organized, and integrity-checked
- [ ] Data validation rules implemented and passing
- [ ] EDA report completed (Section 7 plots)
- [ ] Image and text preprocessing implemented correctly
- [ ] PostgreSQL running with the defined schema
- [ ] pgvector extension installed and HNSW index built
- [ ] Embedding generation pipeline implemented and run end-to-end
- [ ] Image → image search working
- [ ] Text → image search working
- [ ] Multimodal (image + text) search working
- [ ] Evaluation query set constructed from held-out captions
- [ ] Baseline experiments run (random, CLIP)
- [ ] Model comparison run (CLIP vs. SigLIP)
- [ ] Fine-tuning decision documented (done or explicitly deferred, with reasoning)
- [ ] Re-ranking prototype evaluated (if attempted)
- [ ] FastAPI service implemented with all MVP endpoints
- [ ] Frontend built and connected to the API
- [ ] Unit, integration, and API test suites passing
- [ ] Docker and Docker Compose set up and verified from a clean clone
- [ ] MLflow tracking experiments, parameters, metrics, and artifacts
- [ ] CI/CD pipeline running lint, tests, and Docker build on every PR
- [ ] Monitoring dashboard live (latency, error rate, volume)
- [ ] Security checklist applied (upload limits, validation, secrets, CORS)
- [ ] Deployed to a public, working environment
- [ ] Documentation complete (README, architecture diagram, ADRs)
- [ ] GitHub portfolio presentation finalized (Section 35)
- [ ] CV bullet(s) written and reviewed
- [ ] Interview preparation reviewed (Section 36)